

第七次作业

算法分析题

7-3 随机产生互不相同整数

7-3 试设计一个算法，随机地产生范围在 $1 \sim n$ 的 m 个随机整数，且要求这 m 个随机整数互不相同。

第7-3题答案:

一、算法设计思想

题目要求随机产生范围在 $1 \sim n$ 的 m 个随机整数，并且要求这 m 个整数互不相同。若直接不断随机生成整数并检查是否重复，则当 m 接近 n 时，重复概率较大，可能产生较多无效随机数。

采用 Floyd 随机抽样算法。设集合 S 初始为空，依次令 $j = n - m + 1, n - m + 2, \dots, n$ ，每次从 $1 \sim j$ 中等概率随机产生一个整数 t 。若 t 已经在集合 S 中，则把 j 加入 S ；否则把 t 加入 S 。循环结束后，集合 S 中恰好有 m 个互不相同的整数。

该算法的核心思想是：第 j 轮保证从 $1 \sim j$ 中为集合新增一个此前未出现的元素。若随机数 t 未出现，则直接选择 t ；若 t 已出现，则用当前最大候选数 j 替代。由于 j 在本轮前尚未进入抽样区间的最终新增位置，因此不会造成重复。

二、算法分析

1. 正确性分析：

对循环轮数进行归纳。第 j 轮开始前，集合中已选元素均互不相同。若随机得到的 t 不在集合中，则加入 t 后集合仍无重复；若 t 已在集合中，则加入 j 。由于循环按 $j = n - m + 1$ 到 n 递增进行，在当前轮中 j 是本轮新增候选上界，若 t 已被选择，则用 j 替换不会与已有元素冲突。每轮恰好增加一个新元素，经过 m 轮后集合大小为 m ，且所有元素均在 $1 \sim n$ 范围内，所以算法能产生 m 个互不相同的随机整数。

2. 时间复杂度分析：

循环执行 m 次。若用哈希表判断元素是否存在，每次查找和插入的期望时间为 $O(1)$ ，因此总时间复杂度为 $O(m)$ 。

3. 空间复杂度分析：

需要保存最终的 m 个整数，空间复杂度为 $O(m)$ 。

三、代码:

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main() {
5      ios::sync_with_stdio(false);
6      cin.tie(nullptr);
7      int n, m;
8      if (!(cin >> n >> m)) return 0;
9      if (m < 0 || m > n) return 0;
10     mt19937
11     rng((unsigned)chrono::steady_clock::now().time_since_epoch().count());
12     unordered_set<int> S;
13     for (int j = n - m + 1; j ≤ n; j++) {
14         uniform_int_distribution<int> dist(1, j);
15         int t = dist(rng);
16         if (S.count(t)) S.insert(j);
17         else S.insert(t);
18     }
19     vector<int> ans(S.begin(), S.end());
20     sort(ans.begin(), ans.end());
21     for (int i = 0; i < (int)ans.size(); i++)
22         cout << ans[i] << " \n"[i + 1 == (int)ans.size()];
23     return 0;
24 }
```

7-4 集合元素重复抽取问题

7-4 设 X 是含有 n 个元素的集合, 从 X 中均匀地选取元素。设第 k 次选取时首次出现重复。

(1) 试证明当 n 充分大时, k 的期望值为 $\beta\sqrt{n}$ 。其中, $\beta\sqrt{\pi/2}=1.253$ 。

(2) 由此设计一个计算给定集合 X 中元素个数的概率算法。

第7-4题答案:

一、算法设计思想

设集合 X 中共有 n 个元素, 每次从 X 中均匀随机选取一个元素, 允许重复。设第 k 次选取时首次出现重复, 要求分析 n 充分大时 k 的期望数量级, 并设计估计集合大小的随机算法。

当已抽取 $k-1$ 次且没有重复时，前 $k-1$ 次必然是互不相同的。第 k 次出现重复的概率为 $(k-1)/n$ 。因此，直到首次重复所需的抽取次数与生日悖论相同，其期望量级为 $\Theta(\sqrt{n})$ 。更精确地说，首次重复次数的期望近似为 $\sqrt{\pi n/2}$ 。

据此可设计集合大小估计算法：反复进行若干轮随机抽取实验，每轮记录首次重复时的抽取次数 k ，取平均值 $\bar{k}$ ，再由 $\bar{k} \approx \sqrt{\pi n/2}$ 得到估计值

$$n \approx \frac{2\bar{k}^2}{\pi}.$$

二、算法分析

1. 正确性分析：

前 k 次抽取均不重复的概率为

$$\frac{n(n-1)(n-2)\cdots(n-k+1)}{n^k}.$$

当 n 较大且 k 相比 n 较小时，有近似

$$\prod_{i=0}^{k-1} \left(1 - \frac{i}{n}\right) \approx e^{-k(k-1)/(2n)}.$$

当 k 达到 $\sqrt{n}$ 量级时，该概率开始明显下降，因此首次重复出现的期望次数为 $\Theta(\sqrt{n})$ 。利用多次独立实验求平均，可以降低随机误差，因此可用首次重复次数估计集合大小。

2. 时间复杂度分析：

单轮实验的期望抽取次数为 $O(\sqrt{n})$ 。若重复实验 T 轮，总时间复杂度为 $O(T\sqrt{n})$ 。

3. 空间复杂度分析：

每轮实验需要记录已出现元素，最多记录 $O(\sqrt{n})$ 个元素，空间复杂度为 $O(\sqrt{n})$ ；若直接用布尔数组标记全集，则空间复杂度为 $O(n)$ 。

三、代码：

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main() {
5      ios::sync_with_stdio(false);
6      cin.tie(nullptr);
7      int n; long long T = 10000;
8      if (!(cin >> n)) return 0;
9      if (cin.peek() != EOF) cin >> T;
10     mt19937
        rng((unsigned)chrono::steady_clock::now().time_since_epoch().count());
```

```

11     uniform_int_distribution<int> dist(1, n);
12     long double sum = 0;
13     for (long long tc = 0; tc < T; tc++) {
14         vector<int> vis(n + 1, 0);
15         int k = 0;
16         while (true) {
17             int x = dist(rng);
18             k++;
19             if (vis[x]) break;
20             vis[x] = 1;
21         }
22         sum += k;
23     }
24     long double avg = sum / T;
25     long double estimate = 2.0L * avg * avg / acosl(-1.0L);
26     cout.setf(ios::fixed); cout << setprecision(6);
27     cout << "average_k = " << (double)avg << '\n';
28     cout << "estimate_n = " << (double)estimate << '\n';
29     return 0;
30 }

```

7-5 生日相同概率计算

7-5 试设计一个随机化算法计算 $365!/340!365^{25}$ ，并精确到 4 位有效数字。

第7-5题答案:

一、算法设计思想

题目要求随机化算法计算 $365!/340!365^{25}$ ，并精确到 4 位有效数字。该式可写为

$$\frac{365!}{340!365^{25}} = \frac{365 \cdot 364 \cdots 341}{365^{25}},$$

表示 25 个人生日互不相同的概率。因此至少有两人生日相同的概率为

$$1 - \frac{365!}{340!365^{25}}.$$

利用乘法逐项计算避免直接计算巨大阶乘：从 $i = 0$ 到 24 依次乘以 $(365 - i)/365$ 得到生日互异概率，再用 1 减去该值即可。

二、算法分析

1. 正确性分析:

第 1 个人生日可任意选择, 概率为 $365/365$; 第 2 个人生日与第 1 个人不同的概率为 $364/365$; 依次类推, 第 25 个人与前 24 人都不同的概率为 $341/365$ 。因此 25 人生日互不相同的概率正是

$$\prod_{i=0}^{24} \frac{365-i}{365} = \frac{365!}{340!365^{25}}.$$

其补事件即至少两人生日相同, 所以结果为 1 减去上述乘积。

2. 时间复杂度分析:

只需进行 25 次乘法, 时间复杂度为 $O(1)$ 。推广到 k 个人时, 时间复杂度为 $O(k)$ 。

3. 空间复杂度分析:

只使用常数个变量, 空间复杂度为 $O(1)$ 。

三、代码:

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main() {
5      long double ans = 1.0L;
6      int n = 365, k = 25;
7      for (int i = 0; i < k; i++) ans *= (long double)(n - i) / n;
8      long double p = 1 - ans;
9      cout.setf(ios::fixed); cout << setprecision(4) << (double)p << '\n';
10     return 0;
11 }
```

7-9 n 后问题有解性

7-9 如果对于某个 n 值, n 后问题无解, 则算法将陷入死循环。

(1) 证明或否定下述论断: 对于 $n \geq 4$, n 后问题有解。

(2) 是否存在正数 δ , 使得对所有 $n \geq 4$ 算法成功的概率至少是 δ ?

第7-9题答案:

一、算法设计思想

n 后问题要求在 $n \times n$ 棋盘上放置 n 个皇后，使任意两个皇后不在同一行、同一列和同一条对角线上。题目给出结论：当 $n = 2$ 或 $n = 3$ 时无解；对于 $n \geq 4$ 时有解，并要求说明随机算法成功概率是否至少为 $\varepsilon > 0$ 。

对于有解性，可采用回溯算法验证：按行放置皇后，每一行选择一个未被攻击的列。若所有行均成功放置，则得到一个解；若某行无合法列，则回溯。该算法可以完整搜索所有可能方案，因此能够证明 $n = 2, 3$ 无解，也可构造 $n \geq 4$ 的解。

对于随机算法，若在每行从所有合法列中随机选择一个继续搜索，则只要存在至少一个完整解，随机过程沿着该解选择的概率就是正数。由于每一步可选列数最多为 n ，沿某一个固定解走到叶结点的概率至少为 $1/n^n$ ，故对于固定 $n \geq 4$ ，算法一次成功概率至少存在一个正下界。但该下界通常依赖于 n ，例如可粗略估计为 $1/n^n$ ，因此不能推出存在与 n 无关的统一正常数 δ 。

二、算法分析

1. 正确性分析：

回溯算法逐行放置皇后，任意时刻已放置的皇后均满足互不攻击。每次尝试某列前检查列冲突和两条对角线冲突，因此加入新皇后后仍保持合法性。若算法到达第 n 行之后，说明 n 个皇后全部合法放置，得到可行解；若搜索完所有分支仍无解，则不存在任何合法放置方案。由此可判断 $n = 2, 3$ 无解，并可在 $n \geq 4$ 时找到可行解。

2. 时间复杂度分析：

最坏情况下搜索所有列排列，复杂度为 $O(n!)$ ；每次判断合法性若直接扫描前面行，需要 $O(n)$ ，总复杂度为 $O(n \cdot n!)$ 。若用列数组和对角线数组标记，可将每次判断降为 $O(1)$ 。

3. 空间复杂度分析：

递归深度为 n ，保存每行皇后列位置需要 $O(n)$ 空间，因此空间复杂度为 $O(n)$ 。

三、代码：

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int n;
5  vector<int> col;
6  bool ok(int r, int c) {
7      for (int i = 0; i < r; i++)
8          if (col[i] == c || abs(i - r) == abs(col[i] - c)) return false;
9      return true;
10 }
11 bool dfs(int r) {
12     if (r == n) return true;
13     for (int c = 0; c < n; c++) if (ok(r, c)) {
14         col[r] = c;
```

```

15         if (dfs(r + 1)) return true;
16         col[r] = -1;
17     }
18     return false;
19 }
20 int main() {
21     ios::sync_with_stdio(false);
22     cin.tie(nullptr);
23     cin >> n;
24     col.assign(n, -1);
25     if (dfs(0)) {
26         cout << "YES\n";
27         for (int i = 0; i < n; i++) cout << col[i] + 1 << " \n"[i + 1 == n];
28     } else cout << "NO\n";
29     return 0;
30 }

```

7-12 蒙特卡罗算法正确率放大

7-12 设 $mc(x)$ 是一致的 75% 正确的蒙特卡罗算法，考虑下面的算法：

```

mc3(x) {
    int t, u, v;
    t = mc(x);
    u = mc(x);
    v = mc(x);
    if ((t == u) || (t == v))
        return t;
    return v;
}

```

- (1) 试证明上述算法 $mc3(x)$ 是一致的 $27/32$ 正确的算法，因此是 84% 正确的。
- (2) 试证明如果 $mc(x)$ 不是一致的，则 $mc3(x)$ 的正确率有可能低于 71%。

第7-12题答案:

一、算法设计思想

设 $mc(x)$ 是一致的 75% 正确的蒙特卡罗算法。考虑算法 $mc3(x)$ ：独立调用三次 $mc(x)$ ，得到 t, u, v ，若 t 与 u 或 v 相同，则输出 t ，否则输出 v 。该算法实质上是三次独立运行后的多数表决。

当 $mc(x)$ 是一致算法时，每次运行正确概率为 $3/4$ ，错误概率为 $1/4$ ，且错误方向一致。因此 $mc3(x)$ 输出错误当且仅当三次调用中至少两次错误。其错误概率为

$$\binom{3}{2} \left(\frac{1}{4}\right)^2 \left(\frac{3}{4}\right) + \left(\frac{1}{4}\right)^3 = \frac{10}{64} = \frac{5}{32}.$$

所以正确概率为

$$1 - \frac{5}{32} = \frac{27}{32}.$$

若 $mc(x)$ 不是一致算法，错误方向可能不固定，多数表决无法保证错误相互抵消，甚至可能在某些情况下使正确率低于 71%。

二、算法分析

1. 正确性分析：

对一致蒙特卡罗算法而言，三次独立运行中只要至少两次正确，多数表决结果就正确；只有至少两次错误时才输出错误。由二项分布可得错误概率为 $5/32$ ，因此正确概率为 $27/32$ ，约为 84.375%，可证明 $mc3(x)$ 是一致的 $27/32$ 正确算法。

若 $mc(x)$ 不是一致算法，错误输出可能有多个方向。三次结果之间的相等关系不再等价于“多数正确”，从而可能造成正确答案被错误结果覆盖，故正确率可能低于 71%。

2. 时间复杂度分析：

设一次 $mc(x)$ 的时间复杂度为 $T(n)$ ，则 $mc3(x)$ 调用三次，时间复杂度为 $O(T(n))$ ，常数为 3。

3. 空间复杂度分析：

只需保存三次调用结果，空间复杂度为 $O(1)$ 。

7-14 拉斯维加斯算法构造

7-14 设算法 A 和 B 是解同一判定问题的两个有效的蒙特卡罗算法。算法 A 是 p 正确偏真算法，算法 B 是 q 正确偏假算法。试利用这两个算法设计一个解同一问题的拉斯维加斯算法，并使所得到的算法对任何实例的成功率尽可能高。

第7-14题答案：

一、算法设计思想

设算法 A 和 B 是解同一判定问题的两个有效蒙特卡罗算法，其中 A 是 p 正确偏真算法，B 是 q 正确偏假算法。偏真算法表示当正确答案为真时一定能保证输出真，错误只可能发生在答案为假时；偏假算法表示当正确答案为假时一定能保证输出假，错误只可能发生在答案为真时。

要构造拉斯维加斯算法，需要保证算法一旦输出结果，则结果一定正确。可采用“双算法相互验证”的思想：反复运行 A 和 B ，若二者输出一致，则该结果可信并输出；若二者输出不一致，则说明至少有一个算法产生了不可靠输出，此时不输出结果，重新随机运行。

若问题本身还提供多项式时间验证器，则可以进一步在二者一致时调用验证器确认答案。这样算法不会输出错误答案，只是运行次数为随机变量，符合拉斯维加斯算法“永不出错、运行时间随机”的特点。

二、算法分析

1. 正确性分析：

当真实答案为真时，偏真算法 A 必定输出真，若 B 也输出真，则二者一致且输出真为正确答案；若 B 输出假，则二者不一致，算法拒绝输出并重新运行。当真实答案为假时，偏假算法 B 必定输出假，若 A 也输出假，则二者一致且输出假为正确答案；若 A 输出真，则二者不一致，算法重新运行。因此算法只在二者给出共同正确答案时才停止，不会输出错误结果。

单轮停止概率至少与两个算法在当前真实答案下同时给出可靠结果有关。若 A 、 B 独立运行，则当答案为真时停止概率不低于 q ，当答案为假时停止概率不低于 p ，因此期望重复次数有限，为拉斯维加斯算法。

2. 时间复杂度分析：

设 A 、 B 单次运行时间分别为 $T_A(n)$ 、 $T_B(n)$ ，单轮时间为 $O(T_A(n) + T_B(n))$ 。若单轮成功停止概率为 s ，则期望轮数为 $1/s$ ，期望时间复杂度为

$$O\left(\frac{T_A(n) + T_B(n)}{s}\right).$$

3. 空间复杂度分析：

只需保存两个算法的输出和少量控制变量，除算法自身空间外，额外空间复杂度为 $O(1)$ 。

算法实现题

7-3 集合相等问题

7-3 集合相等问题。

问题描述：给定两个集合 S 和 T ，试设计一个判定 S 和 T 是否相等的蒙特卡罗算法。

算法设计：设计一个拉斯维加斯算法，对于给定的集合 S 和 T ，判定其是否相等。

数据输入：由文件 input.txt 给出输入数据。第 1 行有 1 个正整数 n ，表示集合的大小。接下来的 2 行，每行有 n 个正整数，分别表示集合 S 和 T 中的元素。

结果输出：将计算结果输出到文件 output.txt。若集合 S 和 T 相等则输出“YES”，否则输出“NO”。

输入文件示例

input.txt

3

2 3 7

7 2 3

输出文件示例

output.txt

YES

第7-3题答案:

一、算法设计思想

给定两个集合 S 和 T ，要求判断二者是否相等。若直接排序比较，时间复杂度为 $O(n \log n)$ 。为了构造蒙特卡罗算法，可使用随机多项式指纹：随机选取一个数 a ，分别计算

$$F(S) = \sum_{x \in S} a^x \pmod{M}, \quad F(T) = \sum_{x \in T} a^x \pmod{M}.$$

若 $S = T$ ，则两式必然相等；若 $S \neq T$ ，则二者相等只可能是随机碰撞。为进一步降低错误概率，同时使用两个大素数模数进行指纹比较。

该算法属于蒙特卡罗算法：当判定为不相等时一定正确；当判定为相等时存在极小碰撞概率。

二、算法分析

1. 正确性分析：

若 $S = T$ ，两个集合包含完全相同的元素，因此在任意模数下指纹值完全相同，算法一定输出 **YES**。若 $S \neq T$ ，则差多项式不为零。随机选择 a 后，非零多项式在模素数域中取零的概率受到多项式次数限制；使用两个不同大素数模数后，误判概率进一步降低。因此算法满足蒙特卡罗判定要求。

2. 时间复杂度分析：

对每个元素进行两次快速幂计算。若元素大小为 V ，每次快速幂耗时 $O(\log V)$ ，总时间复杂度为 $O(n \log V)$ 。在整数范围较小时，也可预处理幂值将时间降为 $O(n)$ 。

3. 空间复杂度分析：

除输入数组外，只需常数个哈希值和随机数，额外空间复杂度为 $O(1)$ 。

三、代码:

```
1  #include <bits/stdc++.h>
2  using namespace std;
3  using ull = unsigned long long;
4
5  const ull MOD1 = 1000000007ULL;
6  const ull MOD2 = 1000000009ULL;
7
8  ull mod_pow(ull a, ull b, ull mod) {
9      ull r = 1;
10     while (b) {
11         if (b & 1) r = (__uint128_t)r * a % mod;
12         a = (__uint128_t)a * a % mod;
13         b >>= 1;
14     }
15     return r;
16 }
17
18 int main() {
19     ios::sync_with_stdio(false);
20     cin.tie(nullptr);
21     int n;
22     if (!(cin >> n)) return 0;
23     vector<long long> S(n), T(n);
24     for (int i = 0; i < n; i++) cin >> S[i];
25     for (int i = 0; i < n; i++) cin >> T[i];
26     mt19937_64
27     rng((unsigned)chrono::steady_clock::now().time_since_epoch().count());
28     ull a1 = rng() % (MOD1 - 2) + 2, a2 = rng() % (MOD2 - 2) + 2;
29     ull hs1 = 0, ht1 = 0, hs2 = 0, ht2 = 0;
30     for (int i = 0; i < n; i++) {
31         ull x = (S[i] % (long long)MOD1 + MOD1) % MOD1;
32         ull y = (T[i] % (long long)MOD1 + MOD1) % MOD1;
33         hs1 = (hs1 + mod_pow(a1, x, MOD1)) % MOD1;
34         ht1 = (ht1 + mod_pow(a1, y, MOD1)) % MOD1;
35         x = (S[i] % (long long)MOD2 + MOD2) % MOD2;
36         y = (T[i] % (long long)MOD2 + MOD2) % MOD2;
37         hs2 = (hs2 + mod_pow(a2, x, MOD2)) % MOD2;
38         ht2 = (ht2 + mod_pow(a2, y, MOD2)) % MOD2;
39     }
40     cout << ((hs1 == ht1 && hs2 == ht2) ? "YES" : "NO") << '\n';
41     return 0;
42 }
```

7-4 逆矩阵问题

7-4 逆矩阵问题。

问题描述：给定两个 $n \times n$ 矩阵 A 和 B ，试设计一个判定 A 和 B 是否互逆的蒙特卡罗算法（算法的计算时间应为 $O(n^2)$ ）。

算法设计：设计一个蒙特卡罗算法，对于给定的矩阵 A 和 B ，判定其是否互逆。

数据输入：由文件 input.txt 给出输入数据。第 1 行有 1 个正整数 n ，表示矩阵 A 和 B 为 $n \times n$ 矩阵。接下来的 $2n$ 行，每行有 n 个实数，分别表示矩阵 A 和 B 中的元素。

结果输出：将计算结果输出到文件 output.txt。若矩阵 A 和 B 互逆则输出“YES”，否则输出“NO”。

输入文件示例

input.txt

3

1 2 3

2 2 3

3 3 3

-1 1 0

1 -2 1

0 1 -0.666667

输出文件示例

output.txt

YES

第7-4题答案:

一、算法设计思想

给定两个 $n \times n$ 矩阵 A 和 B ，要求判断二者是否互逆，即判断 $AB = I$ 。直接矩阵乘法需要 $O(n^3)$ 时间。可采用 Freivalds 随机检验思想：随机生成一个 0/1 向量 r ，检查

$$A(Br) = r$$

是否成立。

若 $AB = I$ ，则对任意向量 r 都有 $A(Br) = r$ ，算法一定输出 YES。若 $AB \neq I$ ，记 $D = AB - I$ ，则存在随机向量 r 使 $Dr \neq 0$ 的概率至少为 $1/2$ 。重复测试 k 轮后，误判概率不超过 2^{-k} 。

由于输入样例中含有浮点数，程序采用 double 存储矩阵元素，并使用 EPS 控制浮点误差。

二、算法分析

1. 正确性分析：

当 $AB = I$ 时， $A(Br) = Ir = r$ ，每轮检验均通过，算法一定输出 YES。当 $AB \neq I$ 时，矩阵

$D = AB - I$ 非零，至少存在一行非零。随机 0/1 向量与该非零行点积为 0 的概率至多为 $1/2$ ，因此单轮漏检概率至多为 $1/2$ 。重复 k 轮独立测试后，错误输出 YES 的概率至多为 2^{-k} 。

2. 时间复杂度分析：

每轮先计算 Br ，再计算 $A(Br)$ ，均为矩阵向量乘法，时间复杂度为 $O(n^2)$ 。重复 k 轮，总时间复杂度为 $O(kn^2)$ ，当 k 为常数时为 $O(n^2)$ 。

3. 空间复杂度分析：

需要保存随机向量、中间向量和结果向量，额外空间复杂度为 $O(n)$ ；若计入输入矩阵，空间复杂度为 $O(n^2)$ 。

三、代码：

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  const double EPS = 1e-5;
5
6  int main() {
7      ios::sync_with_stdio(false);
8      cin.tie(nullptr);
9      int n;
10     if (!(cin >> n)) return 0;
11     vector<vector<double>> A(n, vector<double>(n));
12     vector<vector<double>> B(n, vector<double>(n));
13     for (int i = 0; i < n; i++)
14         for (int j = 0; j < n; j++) cin >> A[i][j];
15     for (int i = 0; i < n; i++)
16         for (int j = 0; j < n; j++) cin >> B[i][j];
17
18     mt19937
19     rng((unsigned)chrono::steady_clock::now().time_since_epoch().count());
20     int K = 20;
21     for (int tc = 0; tc < K; tc++) {
22         vector<double> r(n), br(n, 0), abr(n, 0);
23         for (int i = 0; i < n; i++) r[i] = rng() & 1;
24         for (int i = 0; i < n; i++)
25             for (int j = 0; j < n; j++) br[i] += B[i][j] * r[j];
26         for (int i = 0; i < n; i++)
27             for (int j = 0; j < n; j++) abr[i] += A[i][j] * br[j];
28         for (int i = 0; i < n; i++) {
29             if (fabs(abr[i] - r[i]) > EPS) {
30                 cout << "NO\n";
31                 return 0;
32             }
33         }
34     }
```

```
31         }
32     }
33 }
34 cout << "YES\n";
35 return 0;
36 }
```